

Advanced Topics in Automation and Control Engineering

Aerial Robotics Laboratory

Modeling - Part I

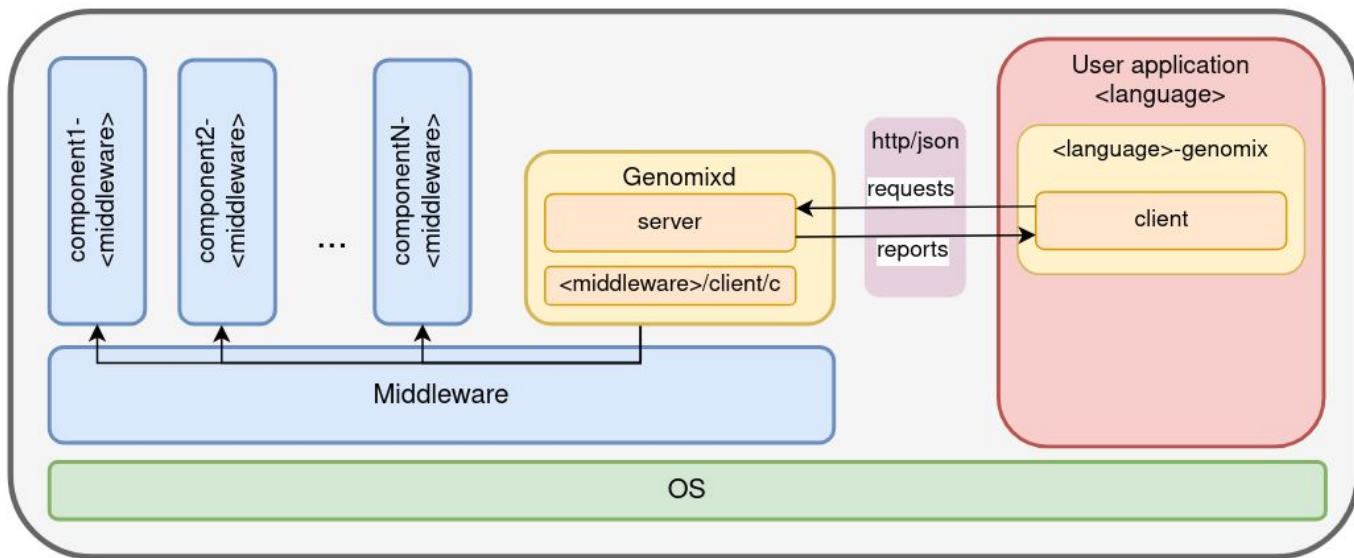
Dr Marco Tognon
Dr Gianluca Corsini



A quick recap first ...

telekyb3 – Modular & middleware-independent framework

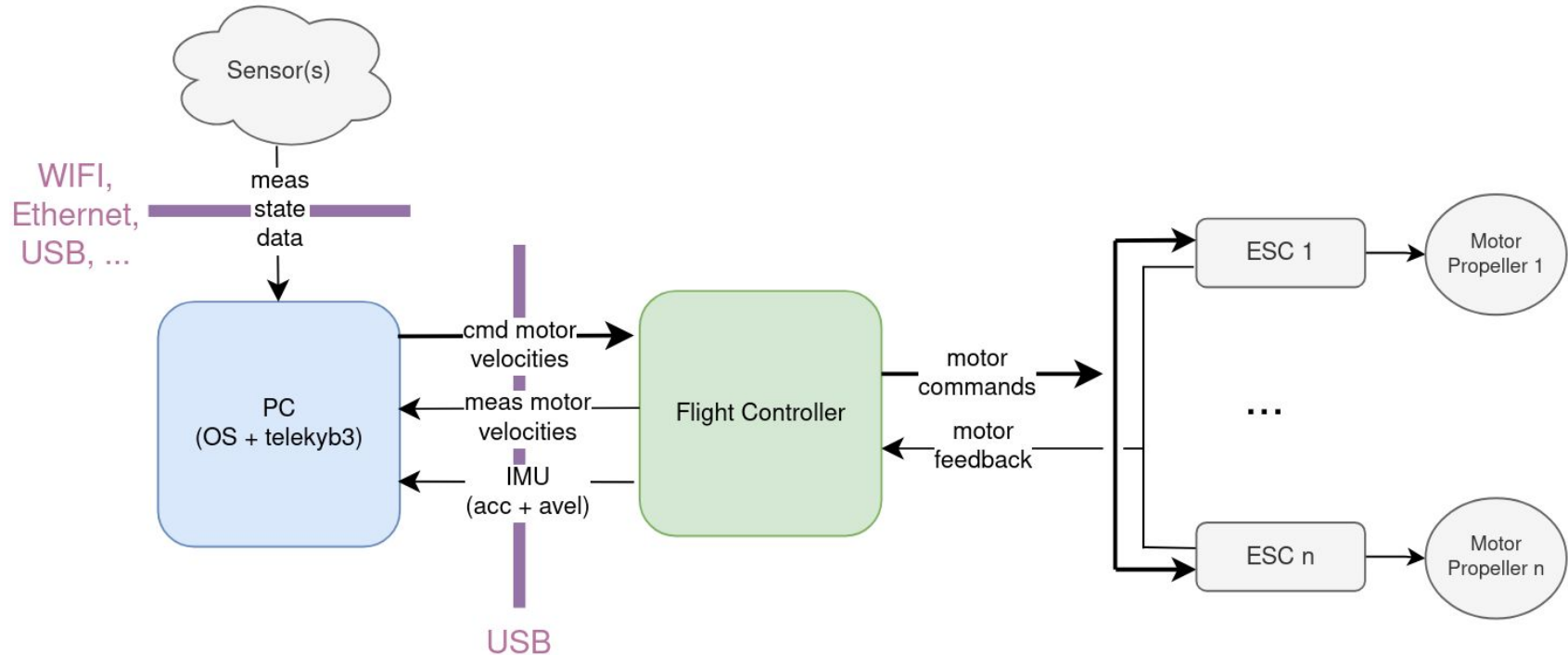
Several components + client (user application, e.g. written in Python)



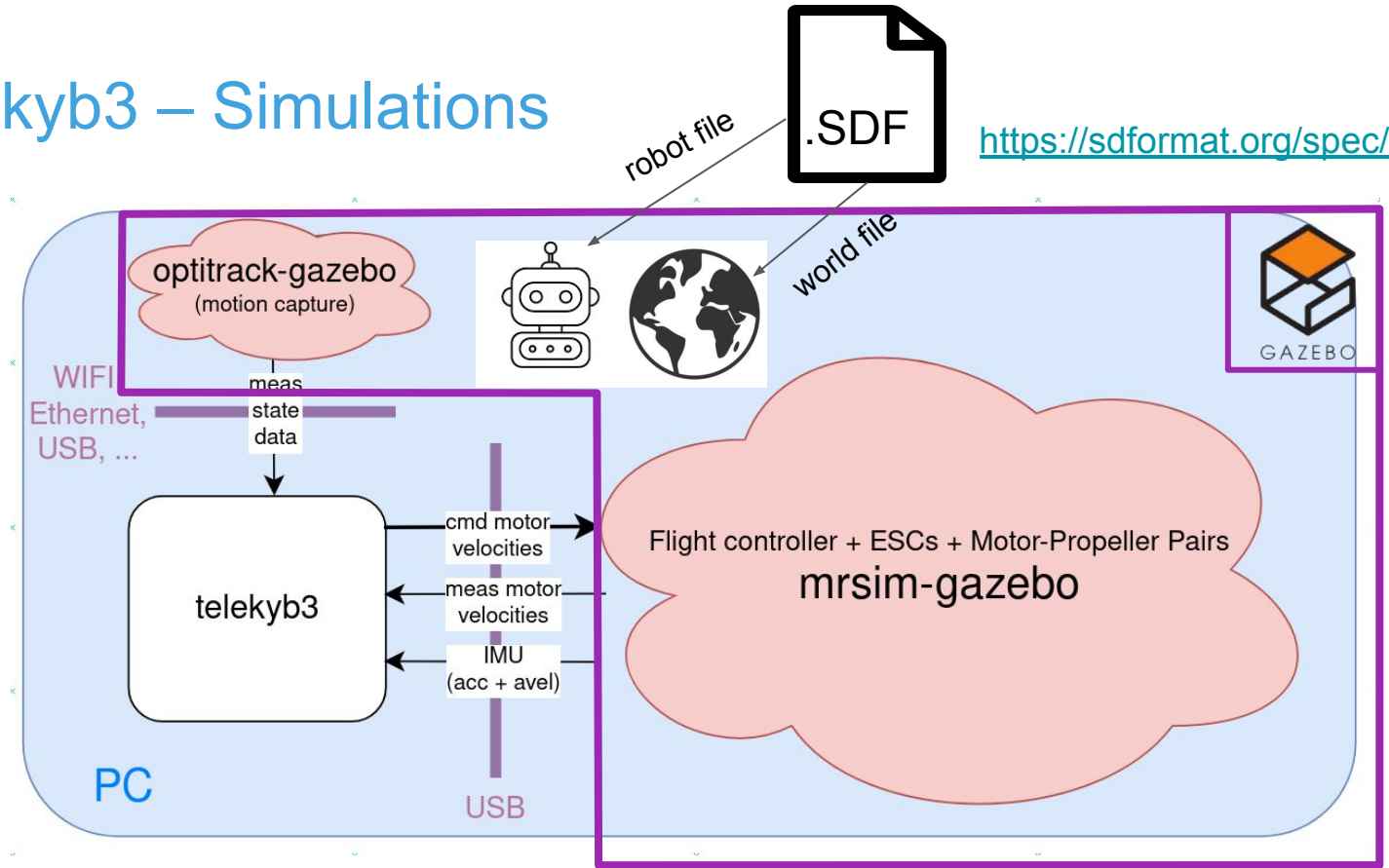
where

- <middleware> = [ROS | pocoLibs | YARP ...]
- <language> = [tcl | python | matlab] , e.g. python-genomix

telekyb3 – Architecture

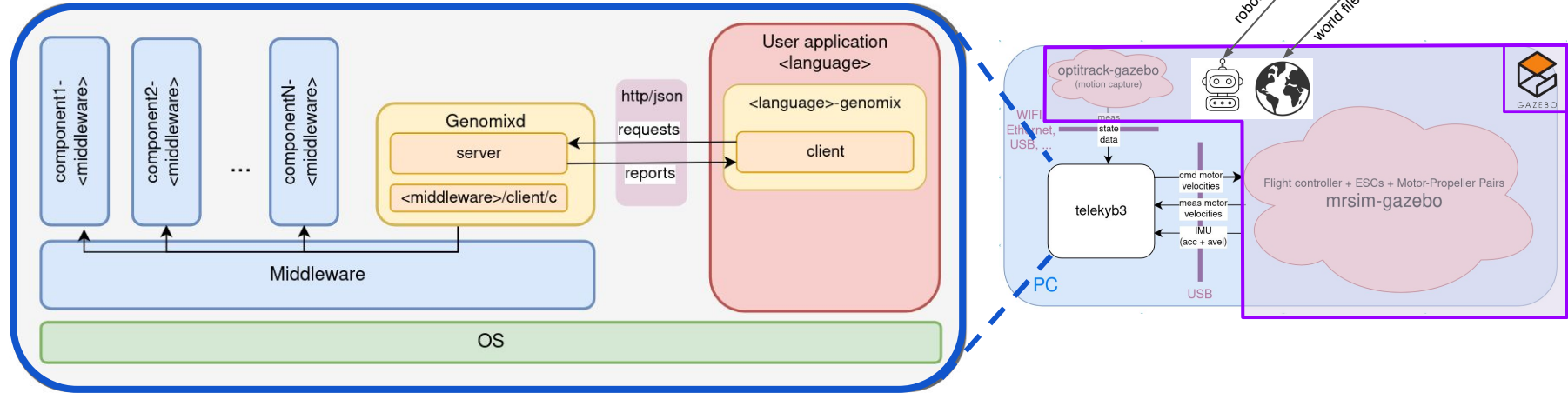


telekyb3 – Simulations



telekyb3 – Simulations

Detailed overview

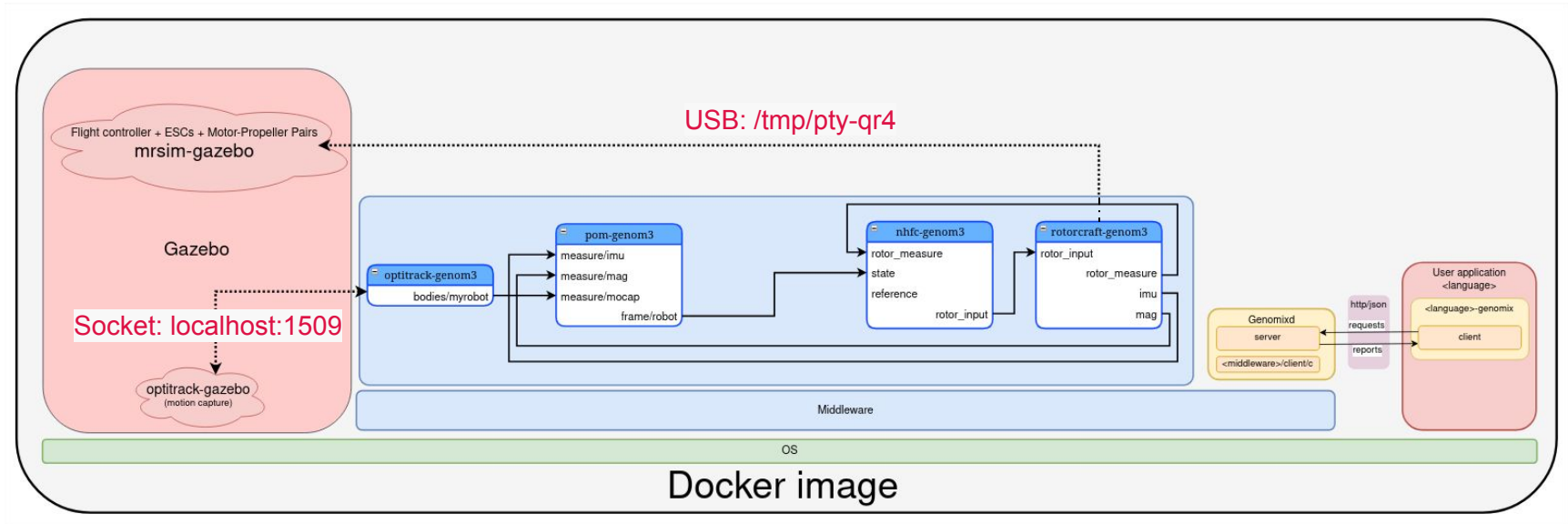


where

- `<middleware>` = `[ROS | pocoLibs | YARP ...]`
- `<language>` = `[tcl | python | matlab]`, e.g. `python-genomix`

telekyb3 – Starting quadrotor simulation

Overview with all the running GenoM3 components and the required port connections



where

- <middleware> = [pocoLibs](#)
- <language> = python ([python-genomix documentation](#))



Today...

Laboratory topic of today

Week n.	Lectures	Hours	Content
5	08/04/26	3	Introduction
5	09/04/26	2	Lecture + Q&A
6	14/04/26	3	Modeling today
6	15/04/26	3	Trajectory generation
6	16/04/26	2	State estimation
7	21/04/26	3	Solutions review
7	22/04/26	3	Motion control
7	23/04/26	2	Physical interaction

Assignment

1. Run Gazebo simulations for the following robots
 - a. Under-actuated quadrotor
 - b. Under-actuated hexarotor
 - c. Fully-actuated hexarotor
 - d. [Under-actuated octorotor]
 - e. [Fully-actuated octorotor]

2. Realize the following plots for each Gazebo simulation
 - a. Tracking plots (desired vs measured) of i) position [m/s], ii) attitude (Euler) [deg], iii) linear velocities [m/s], iv) angular velocities [deg/s], v) linear accelerations [m/s²]
 - b. Controller errors for i) position [m], ii) attitude errors [deg], iii) linear velocities [m/s], iv) angular velocities [deg/s]
 - c. Controller force [N] and torque [Nm] inputs sent to the robot

Assignment

Waypoints (wp) that each robot shall reach during the simulations:

1. Robot starts in wp0: $x=0$, $y=0$, $z=0$, $yaw=0$
2. After 2s from simulation start, ask for wp1: $x=1$, $y=1$, $z=1$, $yaw=0$
3. After 5s from previous wp, ask for wp2: $x=1$, $y=-1$, $z=1$, $yaw=\pi/2$
4. After 5s from previous wp, ask for wp3 (end): $x=0$, $y=0$, $z=0$, $yaw=0$

TIP: Create function “`simulation`” that executes automatically all required functions for the simulation (setup, start, waypoints, ...) when running the python script for a given robot.

Assignment

Recommended plot layouts and styles.

- Each table is a figure window, and each cell a subplot
- Measured values (current) plotted with continuous lines
- Desired values (reference) plotted with dashed lines
- Use RGB for XYZ axes, respectively
- Set limits along plot y axes (ymin, ymax)
- Start plots from time t=0

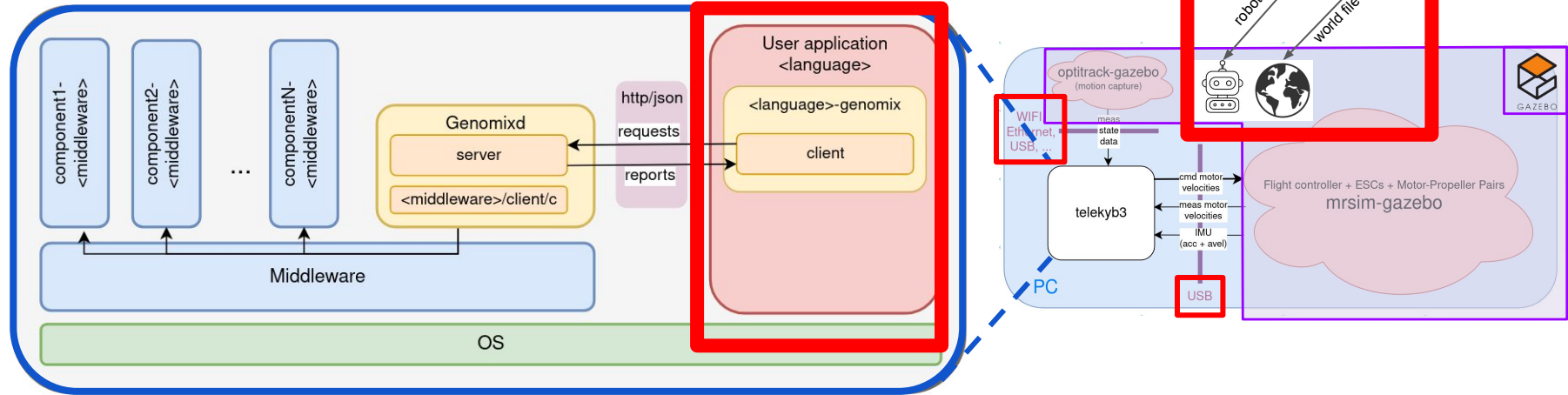
pos (x,y,z)	att (roll,pitch,yaw)
vel (v_x,v_y,v_z)	avel (w_x, w_y, w_z)
acc (a_x, a_y, a_z)	

e_pos (e_x, e_y, e_z)	e_att (e_roll, e_pitch, e_yaw)
e_vel (e_vx, e_vy, e_vz)	e_avel (e_wx, e_wy, e_wz)

f_props for each i in [1,
n_props]

Assignment – Visual representation

In red, the elements to address.



where

- <middleware> = [pocoLibs](#)
- <language> = python ([python-genomix documentation](#))

Assignment – Requirements

Before proceeding with this assignment, we need to discuss about the following topics.

1. SDF Specifications
2. Log files in a GenoM3 component
3. Environment variables
4. Python virtual environments
5. File `.bashrc`
6. Recommended workflow

SDF Specifications

Syntax specifications <https://sdformat.org/spec/>

Documentation to read:

1. Convention → https://sdformat.org/tutorials/specification/convention_tutorial
2. Specifying pose → https://sdformat.org/tutorials/specification/specify_pose
3. Model kinematics → https://sdformat.org/tutorials/specification/spec_model_kinematics
4. Collisions and visuals → https://sdformat.org/tutorials/specification/spec_shapes
5. World files → https://sdformat.org/tutorials/specification/spec_world
6. Composition → <https://sdformat.org/tutorials/specification/composition/>

SDF vs URDF

URDF is not SDF!!!

- URDF → ROS/ROS2
 - Very similar to SDF but different convention to express model kinematics. See example at https://sdformat.org/tutorials/specification/spec_model_kinematics/
- SDF → Gazebo
 - Describes similar information to URDF, but with a different format used in the [Gazebo Simulator](#).
 - Gazebo can read URDFs (internally converted to SDF) and URDF models can be included into SDF files!

Reference: <https://wiki.ros.org/ROS/Patterns/RobotModelling>

Gazebo model and world files in telekyb3

Both installed with the Gazebo plugin [mrsim-gazebo](#)

- Model files at <https://git.openrobots.org/projects/mrsim-gazebo/repository/mrsim-gazebo/revisions/master/show/models>
- World files at <https://git.openrobots.org/projects/mrsim-gazebo/repository/mrsim-gazebo/revisions/master/show/worlds>

In **tk3lab** Docker image, already available.

See the directories models and worlds in /opt/openrobots/share/gazebo/

Log files in GenoM3 components

Each GenoM3 component usually produces log files.

Example of log file: [rotorcrafter-geom3](#)

```
# logged on Tue Jul 15 18:44:20 2025
#
# Rotor commands low-pass filter cut-off frequency
# { fc 0 }
# IMU calibration (0°C average)
#
# [...]
#
# sensor rate
# { imu 1000 mag 0 motor 100 battery 1 }
ts imu_rate mag_rate motor_rate bat imu_temp imu_wx imu_wy imu_wz [...]
1752597860.903931000 1001.06 0 100.781 14.835 nan 0.0929436 0.154479 0.0131851 [...]
1752597860.904879000 1000.37 0 100.781 - nan 0.0518987 0.113548 -0.0213856 [...]
1752597860.905856000 1001.05 0 100.781 - nan 0.0365197 0.114106 -0.0240254 [...]
1752597860.906871000 1000.66 0 100.781 - nan -0.0180517 0.0888274 -0.00156916 [...]
1752597860.907872000 1000.76 0 100.781 - nan -0.0357322 0.0355104 0.0237537 [...]
1752597860.908872000 1000.48 0 100.781 - nan -0.056264 0.0487289 -0.012265 [...]
1752597860.909874000 1000.56 0 100.774 - nan -0.111223 0.0618094 -0.0888731 [...]
1752597860.911874000 1000.07 0 100.668 - nan -0.110873 -0.0913852 -0.0815954 [...]
[...]
```

comments: introduced by '#'

header = first line that is not a comment

record = a line

datapoint = each element in a line

Log files in GenoM3 components

ts “[...] represents the sampling time of the record in seconds since the Unix [epoch](#) (January 1st, 1970). This is convenient to plot synchronized data coming from different log files and components.”

– [telekyb3 plotting documentation](#)

See also: https://en.wikipedia.org/wiki/System_time

TIP: use smallest ts value among all components as t0

```
# ( imu 1000 mag 0 motor 100 battery 1 )
ts imu_rate mag_rate motor_rate bat imu_temp imu_wx imu_wy imu_wz [...]
1752597860.903931000 1001.06 0 100.781 14.835 nan 0.0929436 0.154479 0.0131851 [...]
1752597860.904879000 1000.37 0 100.781 - nan 0.0518987 0.113548 -0.0213856 [...]
1752597860.905856000 1001.05 0 100.781 - nan 0.0365197 0.114106 -0.0240254 [...]
1752597860.906871000 1000.66 0 100.781 - nan -0.0180517 0.0888274 -0.00156916 [...]
1752597860.907872000 1000.76 0 100.781 - nan -0.0357322 0.0355104 0.0237537 [...]
1752597860.908872000 1000.48 0 100.781 - nan -0.056264 0.0487289 -0.012265 [...]
1752597860.909874000 1000.56 0 100.774 - nan -0.111223 0.0618094 -0.0888731 [...]
1752597860.911874000 1000.07 0 100.668 - nan -0.110873 -0.0913852 -0.0815954 [...]
[...]
```

Log files in GenoM3 components

How to parse it and realize plots?

Use `Python`, `matplotlib` and `pandas`

Documentation at: <https://git.openrobots.org/projects/telekyb3/pages/software/faq/plot>

In a genomix client, how to...

- ...start a log? Call the activity `log(<filepath>)`
- ...stop a log? Call the activity `log_stop()`

See the documentation of each component, at

https://git.openrobots.org/projects/<COMPONENT_NAME>-genom3/pages/README

For instance, if `<COMPONENT_NAME>=rotorcrafft`, then

<https://git.openrobots.org/projects/rotorcrafft-genom3/pages/README>

Log files in GenoM3 components

Exceptions may exist!

Some components may have additional services for logging extra data.

For instance, the GenoM3 component [pom-genom3](#) has also [log_measurements](#) and [log_state](#).

The former service ([log_measurements](#)) logs all the measurements fed to the UKF filter implemented by that component, while the latter ([log_state](#)) is used to log the internal state of the filter.

Good to know, but not needed now ;)

Log files in GenoM3 components

[optitrack-genom3](#) header explained

- name → name of the tracked object
- ts → timestamp
- x y z roll pitch yaw → 6D pose in world frame, i.e., position [m] + attitude [rad] (Euler, extrinsic XYZ)

Log files in GenoM3 components

[pom-genom3](#) header explained

- `ts` → timestamp
- `i` → intrinsic (0: false → extrinsic → world frame, 1: true → intrinsic → e.g. body frame)
- `posp attp velp avelp accp` → (boolean flag) presence of any related field
- `x y z` → position in world frame [m]
- `roll pitch yaw` → attitude [rad] in world frame (Euler, extrinsic XYZ)
- `vx vy vz` → linear velocities [m/s] in world frame
- `wx wy wz` → angular velocities [rad/s] in world frame
- `ax ay az` → linear accelerations [m/s²] in world frame
- `sx2 sy2 sz2` → covariances position (std²)
- `sr2 sp2 sh2` → covariances attitude
- `svx2 svy2 svz2` → covariances linear velocities
- `swx2 swy2 swz2` → covariances angular velocities
- `sax2 say2 saz2` → covariances linear accelerations

Log files in GenoM3 components

nhfc-genom3 header explained

- `ts` → timestamp
- `fx fy fz tx ty tz` → output forces [N] and torques [N/m] in body frame to control the robot, which are converted to rotor speeds in the output port
- `meas_fx meas_fy meas_fz meas_tx meas_ty meas_tz` → forces [N] and torques [Nm] computed from measured propeller velocities in world frame
- `xd yd zd rolld pitchd yawd vxd vyd vzd wxd wyd wzd axd ayd azd` → desired setpoints for the controller in world frame
- `e_x e_y e_z e_vx e_vy e_vz e_rx e_ry e_rz e_wx e_wy e_wz` → controller errors

Log files in GenoM3 components

[rotorcraft-genom3](#) header explained

- `ts` → timestamp
- `imu_rate mag_rate motor_rate` → frequency rates [Hz] of the flight controller sensors
- `bat imu_temp` → battery level [V] and imu temperature [$^{\circ}\text{C}$]
- `imu_wx imu_wy imu_wz` → filtered gyroscope angular velocities [rad/s] (1st-order low-pass filter)
- `raw_wx raw_wy raw_wz` → raw gyroscope angular velocities [rad/s]
- `imu_ax imu_ay imu_az` → filtered accelerometer linear accelerations [m/s^2] (1st-order low-pass filter)
- `raw_ax raw_ay raw_az` → raw accelerometer linear accelerations [m/s^2]
- `mag_x mag_y mag_z` → filtered magnetometer data
- `raw_mx raw_my raw_mz` → raw magnetometer data

[...]

Log files in GenoM3 components

[rotorcrafft-genom3](#) header explained

[...]

- cmd_v0 cmd_v1 cmd_v2 cmd_v3 cmd_v4 cmd_v5 cmd_v6 cmd_v7 → commanded propeller velocities [Hz] sent to motors
- meas_v0 meas_v1 meas_v2 meas_v3 meas_v4 meas_v5 meas_v6 meas_v7 → measured propeller velocities [Hz] read from motors

Environment variables

“An environment variable is a user-definable value that can affect the way running processes will behave on a computer. Environment variables are part of the environment in which a process runs. For example, a running process can query the value of the TEMP environment variable to discover a suitable location to store temporary files, or the HOME or USERPROFILE variable to find the directory structure owned by the user running the process.”

– Wikipedia https://en.wikipedia.org/wiki/Environment_variable

```
# Example: print the value of the HOME env. variable
$ echo $HOME
/home/username
```

Gazebo environment variables

Gazebo looks for the following environment variables.

- GZ_SIM_SYSTEM_PLUGIN_PATH: lookup directories for plugins
- GZ_SIM_RESOURCE_PATH: lookup directories for models

```
# inside Docker image
```

```
$ echo $GZ_SIM_SYSTEM_PLUGIN_PATH
```

```
/opt/openrobots/lib/gazebo
```

```
$ echo $GZ_SIM_RESOURCE_PATH
```

```
/opt/openrobots/share/gazebo/models
```

NB: This applies only to the [newer releases of Gazebo](#) (e.g., Ionic), and not to the older releases which are nowadays called [Gazebo classic](#)!

Python virtual environment

“[...] A virtual environment is:

- Used to contain a specific Python interpreter and software libraries and binaries. [...] These are by default isolated from software in other virtual environments and Python interpreters and libraries installed in the operating system (usually in `/usr/bin`).
- Contained in a directory, conventionally named `.venv` or `venv`
- Not checked into source control systems such as Git
- Considered as disposable – it should be simple to delete and recreate it from scratch.
- You don't place any project code in the environment
- Not considered as movable or copyable – you just recreate the same environment in the target location”

– Official documentation: <https://docs.python.org/3/library/venv.html>

NB: Starting from Python3.12, `pip install` returns a warning message.

Python virtual environment

```
# inside the Docker image
$ python3 -m virtualenv ~/tk3lab-ws/venv
# every time to enable it, you need to run
$ source ~/tk3lab-ws/venv/bin/activate
# to verify that the (local) Python interpreter is loaded
$ which python3
# which should print something like
/shared-workspace/venv/bin/python3
# from this moment on, you can install a pkg as
$ pip install <pkg> [<pkg2> ... <pkgN>]
# for example, for recommended packages
$ pip install numpy scipy matplotlib
```

Python virtual environment

Create your Python environment in shared workspace once, install the necessary packages, then re-use it (i.e., source it) at every run. After using it, if another pkg is needed, simply install it. And repeat.

```
# every time to enable your Python virtual environment  
$ source ~/tk3lab-ws/venv/bin/activate
```

That's boring!

Solution?

Use file `.bashrc_<your-username>` which is automatically created (if not already there) by any `tk3lab` Docker image!

.bashrc?

“The **.bashrc** file is a *hidden* script file located in a user’s home directory which runs when the user logs in using the bash shell.”

“The contents of .bashrc can be changed to define functions, command aliases, environment variables, and customize the bash shell to the user’s liking.”

References:

- <https://docs.hpc.shef.ac.uk/en/latest/referenceinfo/linux-shell/the-bashrc-file.html#gsc.tab=0>
- <https://unix.stackexchange.com/questions/129143/what-is-the-purpose-of-bashrc-and-how-does-it-work>

.bashrc in tk3lab Docker images

Why not editing the one in the HOME?

Yes...but any change is going to be lost every time the Docker image is closed.

Developers can add changes when building the Docker images, but each user may have different needs!

That's why it is best to personalize your own file in the shared workspace.

Any **tk3lab** image creates one for you to edit in the shared workspace (**tk3lab-ws**) and it is automatically sourced whenever you open a terminal!

NB: This file is hidden thus a file explorer would not show it by default!

Recommended workflow

1. Suggested `tk3lab-ws` folder structure
2. Use any text editor in host machine to edit files in `tk3lab-ws`
3. (Optional) Set up git repository at root level (`tk3lab-ws`), if you are familiar with it. You can push it to GitHub, ...

```
tk3lab-ws/  
├── .bashrc  
├── gazebo/  
│   ├── models/  
│   │   ├── mrsim-hexa-ua/  
│   │   │   ├── model.config  
│   │   │   └── model.sdf  
│   └── worlds/  
│       └── example.world  
├── logs/  
│   └── 01a-model/  
│       ├── hexa-fa/  
│       ├── hexa-ua/  
│       └── quad/  
├── src/  
│   ├── 01-intro/  
│   │   ├── tk3-quadrotor-control.py  
│   │   └── tk3-quadrotor-simulation.sh  
│   └── 01a-model/  
│       ├── hexa-fa/  
│       ├── hexa-ua/  
│       └── quad/  
└── venv
```

Recommended workflow

4. Set as logging directory the folder `${TK3LAB_WS}/logs/<simulation>` where `<simulation>` is a subdirectory related to the simulation of a given robot
5. Source python virtual environment in `tk3lab-ws`
6. Export env. variables for Gazebo for the model and world files in `tk3lab-ws`

for points 5,6

add these lines to your bashrc

```
source ${TK3LAB_WS}/venv/bin/activate
```

```
export GZ_SIM_RESOURCE_PATH=$GZ_SIM_RESOURCE_PATH:${TK3LAB_WS}/gazebo/models
```

```
export GZ_SIM_SYSTEM_PLUGIN_PATH=$GZ_SIM_SYSTEM_PLUGIN_PATH:${TK3LAB_WS}/gazebo/plugins
```

Recommended workflow

7. Check existing environment variables in **tk3lab** Docker image

```
$ env | grep '^TK3'  
TK3LAB_VERSION=0.2  
TK3DOC_EXAMPLES=/opt/openrobots/share/doc/telekyb3/examples  
TK3LAB_GAZEBO_RELEASE=ionic  
TK3LAB_WS=/home/username/tk3lab-ws  
TK3LAB_ARCH=amd64  
TK3DOC_DIR=/opt/openrobots/share/doc/telekyb3
```